Entropy-based Router Consolidation for Cache-Efficient Block-Sparse Language Models

Viktor Jedich

Abstract

Deploying language models in memory-constrained settings requires moving model parameters from host memory at every forward pass. We study SR-Core, a recursive block-sparse architecture in which a shared block bank is accessed repeatedly with routing decided once and reused, giving a hard working-set guarantee of exactly k active blocks per token, independent of bank size or recursion depth. Analysis of earlier free-routing variants reveals a two-phase routing pattern ($J_1 \approx 0.20$; $J_{2\ldots6} \approx 0.984$) and a reuse/novelty tension in which the collapsed reuse pattern is a training artifact (forced diversity reduces loss by 0.104 nats at r=6), motivating the SR-Core design and a router-shaping objective. We introduce an entropy-based consolidation objective that sharpens the pre-selection router distribution, converting the reuse/novelty tension into a controllable cache/locality axis. At $\lambda = 0.003$, the objective simultaneously improves LRU cache efficiency and held-out code generalization relative to the unregularized baseline, replicating across two independent seeds. At $\lambda = 0.005$, cache metrics improve further with seed-dependent quality effects. Offloading simulation confirms structural advantages: sparse SR-Core requires approximately $4.1\times$ fewer bytes per token than dense layer-offloading at realistic cache capacities. Dense models remain the quality upper bound; the contribution is a controllable systems-level degree of freedom within the sparse model family.

1 Introduction

Deploying large language models under memory constraints requires loading model parameters from host memory at each forward pass. For a 7B-parameter model in fp16, streaming roughly 14 GB of weights from host memory at 16 GB/s would impose an idealized lower bound of about 0.9 s/token before compute and overheads — a regime in which memory movement, not arithmetic, is the binding constraint. Quantization compresses individual parameters but preserves the all-weights-per-token access pattern. Mixture-of-Experts routing (Shazeer et al., 2017; Fedus et al., 2022) reduces *which* parameters are active per token, but standard MoE architectures activate different expert sets per layer, accumulating a transfer budget proportional to depth.

We study a complementary structure: a single shared *block bank* that a recursive router accesses repeatedly. At each recursion step r, the router selects a small set of k blocks from a bank of n blocks and applies them to the running hidden state. The key constraint — the **SR-Core** invariant — is that routing happens only at $r = 1$; steps $r = 2, \ldots, R$ reuse the same block selection. The active weight set per token is therefore exactly k blocks, regardless of recursion depth R. We refer to this as the **working set** ($\mathrm{WS} = k$), and it is independent of bank size n and depth R.

This guarantee is architecturally meaningful: at inference time the identity of the k active blocks is known before any matrix multiply is executed, enabling block-granular prefetching. With $n = 64$ blocks and $k = 8$, the working set covers 12.5% of the bank. Offloading simulation on trained models confirms the structural advantage: sparse SR-Core loads approximately $4.1\times$ fewer bytes per token than dense layer-offloading at a realistic LRU cache capacity of $K = 8$ blocks (3,035 vs. 12,348 KB/token at 16 GB/s). This is a simulated bytes-in-motion estimate under an LRU caching model, not a measured latency speedup.

$\mathrm{WS} = k$ is, however, a *necessary* condition for cache efficiency, not a sufficient one. Whether the actual bytes-in-motion are low depends on how the router distributes selections across tokens. If every token activates a different set of k blocks, the effective working set *across a sequence* approaches n, and a small cache will miss on nearly every request. Cache efficiency requires that routing patterns are **locally stable** — that nearby or semantically similar tokens tend to route through overlapping block sets, so that a modest cache serves a disproportionately large fraction of requests.

We show that unconstrained SR-Core training produces a router that is only partially stable, and that a targeted entropy-based objective can systematically move the router along a **cache/locality axis** — trading a small amount of per-token routing diversity for significantly improved block-level cache hit rates. Mild entropy pressure improves held-out code behavior with modest cache gains; stronger pressure consistently improves cache metrics, with quality effects that are seed-dependent. Dense models remain the quality upper bound throughout; the contribution is a controllable systems-level degree of freedom within the sparse model family.

Contributions. We present:

1. An analysis of earlier recursive-routing dynamics that motivated the SR-Core constraint, revealing a stable two-phase structure (token-specific entry; collapsed reusable core) and an inherent reuse/novelty tension.
2. An entropy-based router consolidation objective that converts this tension into a tunable cache/locality axis, with methodologically consistent negative controls.
3. Empirical evaluation on a four-domain pretraining benchmark confirming a reproducible Pareto frontier between cache efficiency and held-out code generalization, replicated across seeds, and bounded above by a dense quality baseline.

2 Architectural Motivation: Reuse, Novelty, and Two-Phase Routing

Before introducing the entropy objective, we document two routing phenomena that motivated the SR-Core design. Both observations come from analysis of earlier recursive-routing variants in which each step r was free to select independently; this per-step freedom is precisely what the SR-Core constraint removes. The findings are reported here as *motivation* for that constraint and for the entropy objective, not as properties of the final SR-Core model (where S_t is identical across all steps by construction).

2.1 Two-Phase Routing Structure

To diagnose the routing dynamics that motivated the SR-Core constraint, we analyzed earlier recursive-routing variants in which each step r independently selected its own block set — the free-routing regime that SR-Core subsequently replaces with a single shared selection. In those variants we measure the **Jaccard similarity** between the selected block sets of consecutive tokens at each recursion step. Formally, for tokens at positions t and $t+1$, the hard-overlap Jaccard at step r is

$$J_r = \frac{|S_r(t) \cap S_r(t+1)|}{|S_r(t) \cup S_r(t+1)|} \tag{1}$$

where $S_r(t) \subseteq \{1, \ldots, n\}$ is the top-k selection at position t, step r.

In trained models we observe a stable two-phase pattern. At the **entry step** ($r = 1$), Jaccard similarity between consecutive tokens is low ($J_1 \approx 0.20$): each token routes through a largely distinct block set, functioning as a token-specific gatekeeper that selects an entry path into the block bank. At all subsequent steps ($r = 2, \ldots, R$), Jaccard similarity collapses to a near-constant high value ($J_{2\ldots6} \approx 0.984$): the model has settled into a reusable core that is shared across most tokens regardless of content.

This structure emerges spontaneously from language modeling loss without any explicit routing supervision. It is architecturally convenient for working-set minimization — a stable resident core reduces cold-start misses — but it also reveals a potential inefficiency: the reusable core may represent over-consolidation rather than learned representational depth.

2.2 Reuse/Novelty Tension

To test whether the high-Jaccard reuse regime is optimal, we conduct **forced-diversity ablations**: at evaluation time, we mask the k blocks selected at step $r-1$ before computing the top-k at step r, forcing the model to route through fresh blocks at each recursive step.

The results are diagnostic. In a model trained without any routing regularization, forced diversity *improves* loss at deep recursion steps: at $r = 6$, forcing diversity reduced loss by approximately 0.104 nats compared to the model's learned routing, which had converged to near-constant block reuse. This means that the high-Jaccard reuse pattern observed in trained models is not the representationally optimal configuration — it is a training artifact. The routing objective (minimize next-token prediction loss) rewards early reuse because it reduces the variance of the block

computation, but this comes at the cost of the incremental representational update that later recursive steps could provide.

These two observations together define the tension: **reuse** (same blocks, fewer cold-start misses, cache-efficient) versus **novelty** (fresh blocks, more incremental representation, potentially better predictions). Router structure is causally relevant: neither unconstrained reuse nor unconstrained diversity is a sufficient operating point. This motivates controlled router shaping — an objective that exposes the reuse/novelty axis as a tunable degree of freedom rather than maximizing either endpoint.

2.3 From Observation to Intervention

These findings motivate a router-shaping objective, but they do not prescribe the form of that objective. One natural candidate is a cross-token similarity penalty (e.g., encouraging lower Jaccard between consecutive tokens' routing decisions). We tested a soft Jaccard objective and found that it is the wrong lever: it acts on pairwise token overlap but does not control the *concentration* of the router's pre-selection probability distribution. In our evaluation, the soft-Jaccard variant increased route diversity (more unique block combinations across the corpus) while simultaneously reducing hard overlap — the opposite of the cache-beneficial consolidation we seek.

The correct intervention point is the **sharpness of the pre-selection distribution** at $r = 1$. A flatter distribution means the router is uncertain which blocks to select; sharpening it means the router assigns higher probability mass to a smaller set of blocks before the top-k cut, producing more consistent routing choices across tokens with similar representations. Entropy minimization acts directly on this quantity.

These observations motivate router-shaping objectives: not to maximize diversity or reuse blindly, but to expose a controllable axis between route diversity and cache locality.

3 Method: Entropy-based Router Consolidation

3.1 SR-Core Architecture

We briefly formalize the architecture to fix notation. A block bank $\mathcal{B} = \{F_1, \ldots, F_n\}$ contains n parameter blocks, each a two-layer MLP with shared input/output dimension d. The model processes each input token through R recursive applications of the bank. At step $r = 1$, a learned router produces a probability distribution over blocks:

$$\mathbf{p}_t = \mathrm{softmax}\left(\mathbf{q}_t^\top \mathbf{K}\right) \in \mathbb{R}^n \tag{2}$$

where $\mathbf{q}_t = W_q \mathbf{h}_{t,0}$ is a query derived from the initial hidden state and $\mathbf{K} \in \mathbb{R}^{n \times d_k}$ is a learned key matrix. The **SR-Core** selection is

$$S_t = \text{top-}k(\mathbf{p}_t), \qquad \mathrm{WS} = |S_t| = k \tag{3}$$

with soft gates $g_{t,b} \propto p_{t,b}$ for $b \in S_t$. The hidden state update at step r is

$$\mathbf{h}_{t,r} = \mathbf{h}_{t,r-1} + \sum_{b \in S_t} g_{t,b} \cdot F_b(\mathbf{h}_{t,r-1}) \tag{4}$$

using the *same* selection S_t for all $r = 1, \ldots, R$. The active working set is therefore exactly k blocks per token, independent of R and n.

Training uses a deep-supervision language modeling loss that supervises the output at every recursion depth, with end-weighted contributions to preserve full-depth quality while allowing intermediate readouts.

3.2 Entropy-based Consolidation Objective

We augment the language modeling loss with an entropy penalty on the pre-selection router distribution:

$$\mathcal{L} = \mathcal{L}_{\mathrm{LM}} + \lambda \cdot \mathcal{H} \tag{5}$$

where

$$\mathcal{H} = \frac{1}{N} \sum_{t=1}^{N} H\left(\mathbf{p}_t^{(1)}\right), \qquad H(\mathbf{p}) = -\sum_{i=1}^{n} p_i \log p_i \tag{6}$$

and $\mathbf{p}_t^{(1)}$ is the router distribution for token t at $r = 1$. The entropy penalty $\mathcal{H}$ encourages the router to assign concentrated probability mass before the top-k selection: lower entropy corresponds to higher confidence in the selected blocks, which in practice produces more consistent routing choices across tokens with similar content and thereby improves cross-token cache hit rates.

Why $r = 1$ only. Routing decisions are made at $r = 1$; steps $r = 2, \ldots, R$ reuse the same block set S_t and do not have a meaningful "unconcentrated" distribution to regularize. Applying the entropy loss at $r = 1$ is therefore the canonical intervention point.

Why pre-top-k. The router distribution $\mathbf{p}_t$ is computed before the top-k cut. Applying entropy regularization at this point shapes the entire selection landscape: the penalty encourages the router to place probability mass on a small subset of blocks before any discretization, rather than penalizing the post-selection gates (which are already sparse by construction). A concentrated pre-selection distribution also improves the router's top-k margin $p_k - p_{k+1}$, stabilizing which blocks fall at the selection boundary.

Distinction from pairwise objectives. We test soft-Jaccard-style overlap objectives as an internal negative control. Such objectives operate on *cross-token* routing similarity: they compare the block selections of consecutive tokens and penalize dissimilarity. This is a different quantity from the *within-token* distribution concentration that entropy targets. In our evaluation, soft-Jaccard objectives increase soft overlap metrics while simultaneously increasing the number of unique routing combinations — the opposite of the cache-beneficial consolidation we seek, and the wrong lever for controlling pre-selection sharpness. Entropy minimization, by acting on the per-token distribution before the top-k cut, produces the qualitatively different effect of a reduced unique routing vocabulary.

Relationship to load balancing. Standard MoE load-balancing losses (Fedus et al., 2022) promote *uniform* block utilization, which is the opposite of entropy minimization. We do not replace load balancing with the entropy objective; the two terms address different failure modes (dead blocks vs. over-diffuse routing) and can be combined. In our experiments we observe that trained SR-Core models already activate all n blocks across a corpus; load balancing is not required as a prerequisite.

Hyperparameter λ. We sweep $\lambda \in \{0.001, 0.003, 0.005, 0.007\}$. The penalty is dimensionally compatible with the language modeling loss (both in nats), making λ a direct ratio between the two terms. Across our sweep, $\lambda = 0.003$ is the most reliable generalization-balanced operating point, with cache and code-generalization improvements that replicate across seeds. $\lambda = 0.005$ is the stronger cache/systems point, delivering larger LRU byte reductions; its quality effects are seed-dependent and should not be characterized as a uniform generalization cost (Section 5).

4 Experimental Setup

Dataset. We train and evaluate on HeteroMini-v1, a four-domain pretraining corpus of approximately 6.6M tokens from Web (FineWeb-Edu), Wikipedia, Code (The Stack, Python), and Literature (Project Gutenberg), tokenized with a shared byte-level BPE vocabulary of 8,000 tokens. Documents are kept contiguous; evaluation uses a matched held-out split with no training overlap.

Model and training. We use SR-Core with $n = 64$ blocks, $k = 8$ active blocks per token, and $R = 6$ recursion steps ($\sim$19M parameters, $d = 256$, block hidden dimension 512). The dense quality baseline is a 24-layer Transformer (DENSE_D24, $\sim$100M parameters). All models use AdamW with a cosine schedule and a deep-supervision LM loss. SR-Core models are trained for 15,000 steps as a shared base, then continued for 2,000 steps with the entropy objective at $\lambda \in \{0.001, 0.003, 0.005, 0.007\}$; the unregularized continuation (CTRL) provides the direct ablation baseline. The dense baseline trains end-to-end for 17,000 steps. Cross-seed replication uses an independently initialized base; we report seed 1 results for $\lambda \in \{0.003, 0.005\}$.

Negative controls. *Softfull* applies a soft Jaccard penalty on router probability vectors across consecutive tokens; *softsharp_a2* adds temperature sharpening ($\alpha = 2$); *reduced_noise* uses lower Gumbel noise ($\sigma = 0.1$). All controls train for 2,000 steps from the shared 15k base.

Evaluation. Routing metrics are measured on 40 held-out batches via `eval_compare.py`: router entropy $H(\mathbf{p})$, unique core combinations, hard-overlap Jaccard, and LRU bytes/token at cache capacities $K \in \{8, 16, 24, 32\}$.

Per-domain held-out loss is measured via paired runs with the dense baseline as a consistent anchor. Offloading simulation uses an LRU cache trace over 1,024 held-out sequences.

5 Results

5.1 Router Consolidation

Figure 1 shows routing metrics as a function of λ. All four panels respond monotonically to entropy pressure across the sweep range $\lambda \in [0, 0.007]$, as summarized in Table 1.

Table 1: Router consolidation metrics across the λ sweep. All values are measured on 40 held-out batches via `eval_compare.py`.

λ	$H(\mathbf{p})$	Unique cores	Hard overlap	K=24 LRU (KB/tok)
0.000 (CTRL)	3.846	25,853	0.251	1,103
0.001	3.815	25,399	0.254	1,098
0.003	3.799	23,077	0.262	1,085
0.005	3.732	21,273	0.266	1,020
0.007	3.647	18,817	0.278	1,009

Router entropy decreases monotonically (more concentrated pre-selection distribution), unique core combinations decrease (fewer distinct routing vocabularies), and hard-overlap Jaccard across consecutive tokens increases (more stable routing paths). The LRU cache metric at $K = 24$ decreases from 1,103 to 1,009 KB/token across the sweep — a 8.5% reduction in simulated bytes-in-motion.

The response is non-uniform across the range. At $\lambda = 0.001$ the effect on routing vocabulary (unique cores) is marginal (-1.8%); the main consolidation occurs between $\lambda = 0.001$ and $\lambda = 0.005$. At $\lambda = 0.007$ the unique-core count continues to decrease, but as Section 5.2 shows, quality effects become less favorable. We identify three qualitative regimes: a *generalization-balanced* region ($\lambda \leq 0.003$), a *cache sweet spot* ($\lambda \approx 0.005$), and a *boundary* ($\lambda = 0.007$) where consolidation is maximal but the generalization picture becomes less reliable.

5.2 Cache–Quality Pareto

Figure 2 plots each model as a point in the (K=24 LRU bytes/token, code-ratio held-out) plane. The *code ratio* is defined as the ratio of held-out code loss to the mean of held-out Web, Wikipedia, and Literature losses; lower values indicate better code generalization relative to the other domains.

λ=0.003 **Pareto-dominates CTRL.** In seed 0, $\lambda = 0.003$ improves K=24 cache efficiency by 1.6% (1,103 $\rightarrow$ 1,085 KB/token) while simultaneously improving the code-generalization ratio from 0.886 to 0.866 ($\Delta = -0.021$). This improvement replicates in seed 1, where the code-ratio improvement is larger ($0.896 \rightarrow 0.822$, $\Delta = -0.074$). λ=0.003 is therefore a Pareto improvement over the unregularized baseline: mild entropy pressure sharpens routing in a way that is simultaneously beneficial for cache locality and code generalization.

λ=0.005**: stronger cache, seed-dependent quality.** $\lambda = 0.005$ achieves the largest consistent cache improvement in the sweep (K=24: 1,103 $\rightarrow$ 1,020 KB/token, -7.5%; K=16: -5.2%). Within-run paired comparisons show opposing signals across seeds: in seed 0, the code-ratio is 0.877 versus the within-run CTRL of 0.859 ($\Delta = +0.018$, slight degradation); in seed 1, the ratio is 0.841 versus within-run CTRL 0.863 ($\Delta = -0.022$, improvement). The Pareto figure uses a single CTRL anchor (0.886) for visual consistency; λ=0.005 (0.877) and λ=0.003 (0.866) both fall below that anchor. The code-quality effect of $\lambda = 0.005$ is seed-dependent and should not be characterized as a consistent cost or benefit.

λ=0.007**: cache-best boundary.** At $\lambda = 0.007$ the cache metric reaches its lowest value in the sweep (K=24: 1,009 KB/token, $\approx$1.1% improvement over λ=0.005), while the code-ratio returns toward CTRL levels (0.884 in seed 0). λ=0.007 is cache-best but the additional gain over λ=0.005 is small relative to the loss of the balanced generalization behavior observed at λ=0.003. We treat λ=0.007 as the boundary point of the controllable axis: further entropy pressure yields diminishing cache returns with less predictable quality effects.

Target-entropy variants. We also evaluate a bounded entropy variant, $\lambda \cdot \mathrm{relu}(H(\mathbf{p}) - H_{\mathrm{target}})$, with targets $H = 3.75$ and $H = 3.70$. Both fall below the Pareto frontier of the unconstrained sweep: the bounded objective

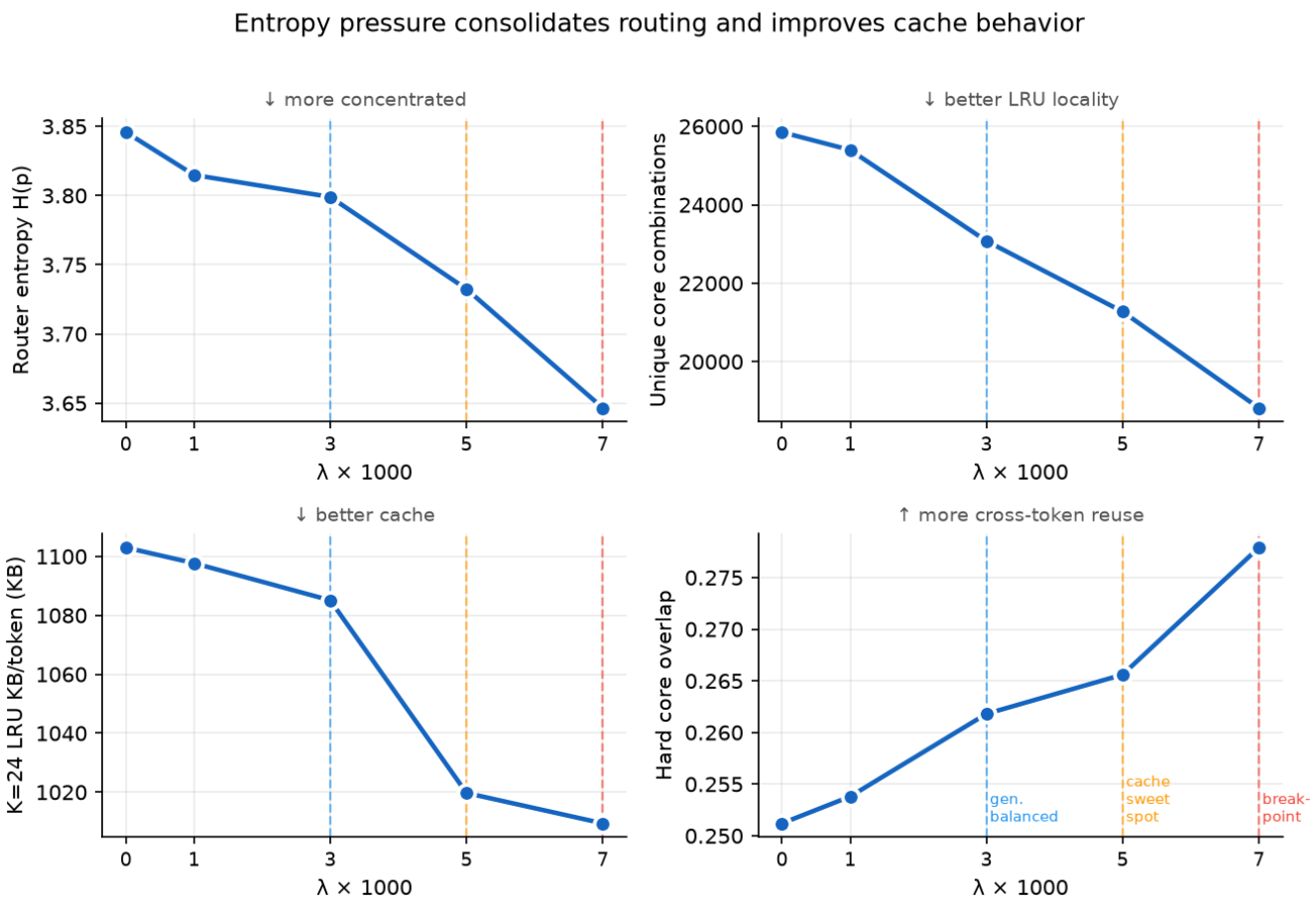

Figure 1: Router consolidation metrics as a function of entropy pressure ($\lambda \times 10^3$). Upper left: router entropy $H(\mathbf{p})$ (lower = more concentrated pre-selection distribution). Upper right: unique core combinations per evaluation run (lower = smaller routing vocabulary). Lower left: K=24 LRU bytes/token (lower = better cache efficiency). Lower right: hard-overlap Jaccard across consecutive tokens (higher = more stable routing paths). All four panels respond monotonically. Dashed vertical lines mark the three qualitative regimes: generalization-balanced (λ=0.003, blue), cache sweet spot (λ=0.005, orange), and boundary (λ=0.007, red).

converges more slowly within the 2,000-step continuation window and does not reach the same cache/quality operating points.

5.3 Dense Quality Baseline

Figure 3 shows held-out loss across all four domains for the dense baseline and three sparse models, evaluated on the same batches. Dense d24 is the quality upper bound in every domain (Table 2).

The dense advantage is largest in Literature (0.55 nats) and smallest in Code (0.24 nats). Entropy-minimized variants do not narrow the dense–sparse quality gap; the objective shapes routing structure without a commensurate language modeling quality gain in paired evaluation. The contribution of entropy minimization is a systems-level degree of freedom, not a path toward dense-equivalent quality.

These results are evaluated with the dense baseline as a consistent anchor (same held-out batches for all models in each paired run). Within-run deltas are reliable; absolute cross-run comparisons should be interpreted with the batch-sampling variance of approximately ± 0.05 nats in mind.

5.4 Negative Controls

The *softfull* objective — a soft-Jaccard penalty encouraging pairwise router similarity across consecutive tokens — moves routing metrics in the opposite direction from entropy minimization. Compared to CTRL, *softfull* increases unique core combinations from 25,684 to 30,246 (+17.8%) and decreases hard-overlap Jaccard from 0.259 to 0.251. Router entropy rises slightly (3.843 → 3.868), and K=24 LRU bytes change by less than 1.4%.

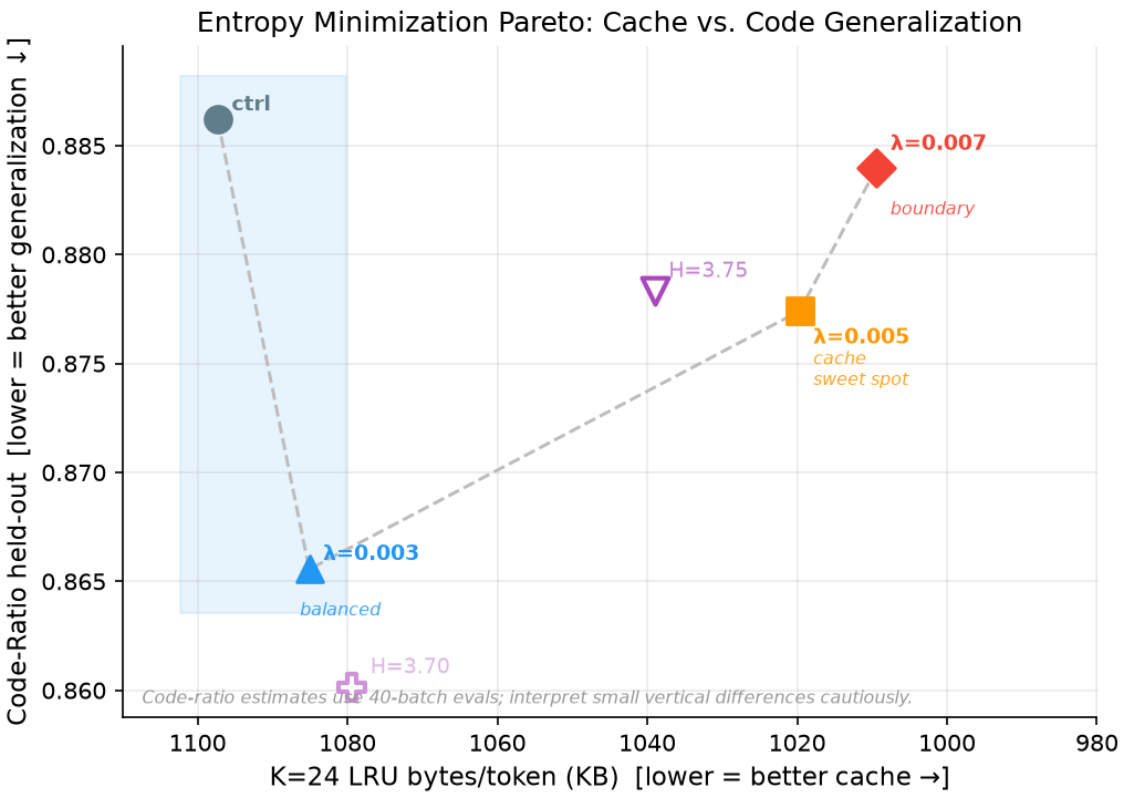

Figure 2: Cache–quality Pareto under entropy pressure. Each point represents a model evaluated on K=24 LRU bytes/token (x-axis; lower = better cache locality, axis reversed) and held-out code-ratio (y-axis; lower = better code generalization). All points share a single CTRL anchor from the λ=0.003 evaluation run for visual consistency. The shaded band highlights the dominance zone of λ=0.003 over CTRL. Target-entropy variants (H=3.75, H=3.70; hollow markers) fall below the Pareto frontier of the unconstrained sweep.

Table 2: Held-out language modeling loss per domain (paired evaluation; same batches per run, dense as consistent anchor). Dense d24 is the quality upper bound in all domains.

Model	Web	Wiki	Code	Lit
Dense d24	5.448	5.456	4.621	4.735
SR-Core CTRL	5.807	5.841	4.856	5.286
+ entmin λ=0.003	5.672	5.688	4.969	5.352
+ entmin λ=0.005	5.887	5.792	4.985	5.419

This confirms the mechanism described in Section 2.3: cross-token similarity objectives act on pairwise overlap but do not control the within-token distribution concentration. The *softfull* result is not a failure of regularization strength — it is a directional failure. Maximizing cross-token similarity can be achieved by the router increasing route diversity (unique routing combinations) while still mapping similar tokens to similar paths on average. Pre-selection entropy is the correct intervention point.

5.5 Offloading Simulation

To bridge the cache-simulation metrics to a bytes-in-motion estimate, we run the LRU offloading simulator on the fully-trained 17k-step checkpoints (Table 3).

At $K = 8$ cached blocks, sparse SR-Core requires $4.1\times$ fewer bytes per token than dense layer-offloading; entropy consolidation at $\lambda = 0.005$ adds a further 2% reduction (3,035 $\rightarrow$ 2,973 KB/token). These are simulated estimates under an LRU caching model at 16 GB/s bandwidth; they do not account for transfer scheduling, compute overlap, or dispatch overhead.

One structural boundary is worth noting: at cache capacities $K \geq n_{\text{layers}} = 24$ for Dense d24, the dense model fits entirely in cache and its bytes-in-motion approach zero, while the sparse model still incurs misses from a bank of

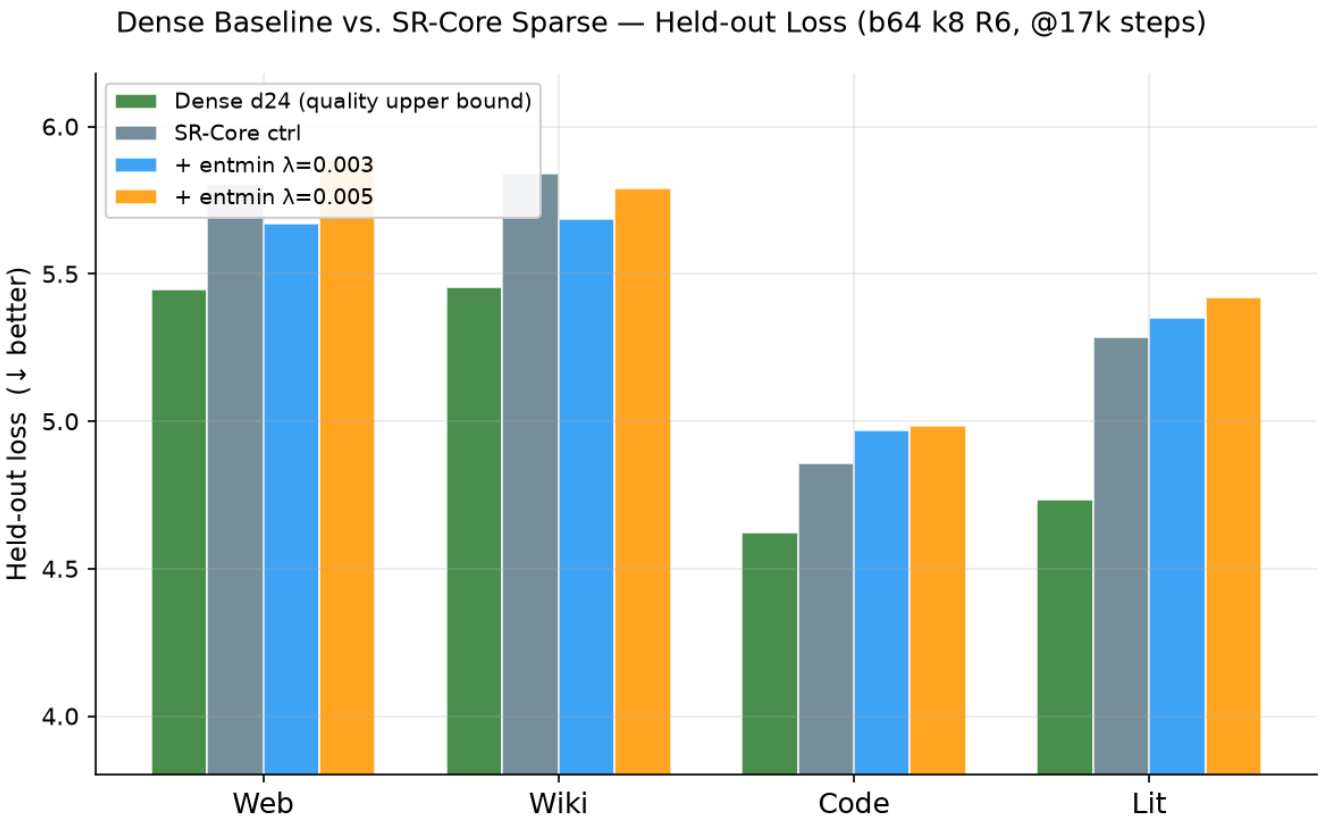

Figure 3: Dense quality baseline vs. SR-Core sparse variants. Held-out language modeling loss across four domains for Dense d24 and three SR-Core sparse models, evaluated on paired held-out batches (dense as consistent anchor per run). Dense d24 is the quality upper bound in every domain. The Δ annotation on the Code bars shows the CTRL–dense gap ($\Delta = 0.24$ nats); Code has the smallest dense–sparse gap and Literature the largest (0.55 nats). Entropy-minimized variants are evaluated for systems behavior (cache, routing), not dense-equivalent quality.

$n = 64$ blocks. We report only $K = 8$ and $K = 16$ in Table 3; the dense crossover occurs outside these reported columns at $K \geq 24$. This capacity crossover is not a contradiction of the sparse working-set result; it reflects that a cache large enough to hold the entire dense model eliminates its transfer cost, an assumption that fails precisely in the memory-constrained deployment regime this work targets.

6 Discussion

What this work shows. SR-Core provides a hard working-set guarantee: exactly k blocks are active per token, regardless of bank size or recursion depth. Entropy-based router consolidation exposes a controllable axis within this fixed working set: increasing λ moves the router toward more concentrated, stable routing paths, reducing simulated bytes-in-motion and unique routing vocabulary. At $\lambda = 0.003$ we observe no measured tradeoff: cache locality and held-out code generalization both improve relative to the unregularized baseline, and the improvement replicates across two independent seeds. At $\lambda = 0.005$ the cache benefit is larger and the quality effect is seed-dependent. Dense models remain the quality upper bound throughout.

What this work does not show. We do not demonstrate wall-clock inference speedup. The dispatch overhead of sparse block selection imposes a CPU-side penalty (approximately $2.8\times$ slower than compute-matched dense in our CPU benchmark), so the bytes-in-motion reduction does not directly translate to latency improvement without hardware co-design. We also do not demonstrate quality parity with dense models, scale the bank beyond $n = 64$ blocks, or evaluate on downstream tasks. The bytes-in-motion estimates are from simulation under idealized LRU caching; real transfer costs depend on block size, DRAM bandwidth, PCIe characteristics, and prefetch scheduling, none of which are accounted for here.

Table 3: Simulated bytes-in-motion under LRU caching at 16 GB/s bandwidth (17k-step checkpoints). We report $K = 8$ and $K = 16$; the dense model crossover occurs at $K \geq 24$ (outside these columns).

Model	K=8 KB/tok	K=16 KB/tok	vs. Dense (K=8)
Dense d24	12,348	12,348	1.00×
SR-Core CTRL	3,035	1,857	0.246×
+ entmin λ=0.003	3,015	1,808	0.244×
+ entmin λ=0.005	2,973	1,747	0.241×

Scope of the entropy objective. Entropy minimization sharpens the pre-selection distribution at $r = 1$; it does not explicitly maximize diversity or cross-token overlap. This means it is not a general solution to the reuse/novelty tension identified in Section 2.2: it moves the router toward the reuse end of the axis in a controlled way, which happens to benefit cache efficiency. Whether a different objective that explicitly targets diversity at deep recursion steps could yield complementary gains remains an open question.

Future directions. The most immediate validation gap is real-latency measurement: an end-to-end offloading prototype (RAM $\rightarrow$ VRAM) would connect the bytes-in-motion simulation to observable tokens/second. Scaling the bank size beyond $n = 64$ is also necessary to test whether the working-set invariant holds at the regime (large n, very small active fraction) where the transfer-reduction ratio is most compelling. On the regularization side, targeted entropy objectives (e.g., penalizing only when entropy exceeds a threshold) and their interaction with load-balancing terms remain unexplored at the trained-model scale.

7 Related Work

Mixture of Experts. Sparse gating routes tokens to expert feed-forward networks (Shazeer et al., 2017; Fedus et al., 2022). Standard MoE architectures activate a different expert subset at each layer, accumulating a transfer budget proportional to depth; SR-Core replaces per-layer routing with a single shared bank and a fixed working set WS $= k$, independent of recursion depth. Load-balancing losses (Lepikhin et al., 2021; Fedus et al., 2022) address expert collapse; the entropy objective addresses over-diffuse routing — a distinct failure mode.

Looped and recurrent depth. The Universal Transformer (Dehghani et al., 2019) and Deep Equilibrium Models (Bai et al., 2019) apply shared parameters repeatedly. SR-Core fixes routing at $r = 1$, giving a predictable active-block set at inference time rather than a variable computation graph — a property these models do not address.

Parameter offloading. FlexGen (Sheng et al., 2023) and DeepSpeed Inference (Aminabadi et al., 2022) optimize layer-offloading schedules for dense transformers; SR-Core reduces *what* is loaded per token. The two approaches are complementary.

Routing regularization. MoE auxiliary objectives include load-balancing losses (Fedus et al., 2022), z-loss (Zoph et al., 2022), and router noise (Shazeer et al., 2017). These target dead experts or training instability; none control the cross-token cache locality that the entropy objective addresses.

Weight sharing. SHA-RNN (Merity, 2019) and related architectures reuse the same weights structurally; SR-Core adds content-dependent routing, keeping the active count fixed at k while varying *which* weights are active per token.

References

Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC'22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

Shaojie Bai, J.Žico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.

Stephen Merity. Single headed attention RNN: Stop thinking with your head. Technical Report arXiv:1911.11423, Salesforce Research, 2019.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning (ICML)*, 2023.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. Technical Report arXiv:2202.08906, Google, 2022.